

Lab Practicum 01: Empirical Parallel Computing

Silicon Profiling, Multi-Core Scaling Limits, and Concurrency Hazards

Instructor: Sufyan bin Uzayr | Duration: 90 Minutes | Weight: 5% of Course Grade

Student Name	Student ID	Host CPU / SoC Model	Operating System & Kernel
Aruzhan Kaparova	230103148	Apple M2 (4P + 4E cores)	macOS Ventura 13,4

TASK 1: Host Silicon Audit & Flynn's Taxonomy Mapping (20 Points)

CPU Model & Architecture	Apple M2 (ARM64 SoC)	Base & Max Boost Clock	Not exposed via sysctl; manufacturer-rated boost ~3.49 GHz
Physical Hardware Cores	8 (4 Performance + 4 Efficiency)	Logical Cores (SMT/Threads)	8 (no SMT/Hyper-Threading)
L1 Data / Instruction Cache	P-core: 128 KB / 192 KB; E-core: 64 KB / 128 KB	L2 Cache (Per Cluster / Total)	P-cluster: 16 MB; E-cluster: 4 MB (Total: 20 MB)
L3 Shared Last-Level Cache	None (Apple Silicon uses a System Level Cache, not exposed via sysctl)	SIMD Vector Extensions	ARM NEON (128-bit), Apple AMX

Q1.1: Flynn's Taxonomy Mapping

- SISD: A single CPU core executes one instruction stream on one set of data. This is typical for sequential code that does not benefit from parallelization.
- SIMD: One instruction operates on multiple data elements simultaneously using the M2's 128-bit ARM NEON SIMD units.
- MIMD: Multiple CPU cores can execute different instruction streams on different data at the same time. My M2 has 8 physical CPU cores (4 P-cores + 4 E-cores).

TASK 2: Multi-Thread Scaling Benchmark & Contention Wall (30 Points)

Workload Algorithm: Prime counting via trial division up to 5,000,000 | Language & Runtime: C, pthreads

Threads (N)	Run 1 (s)	Run 2 (s)	Run 3 (s)	Avg Time TN (s)	Speedup SN=T1/TN	Efficiency EN=SN/N
N = 1	0.5044	0.5056	0.5276	0.5125	1.00×	100.0%

N = 2	0.3139	0.3162	0.3141	0.3147	1.63×	81.4%
N = 4	0.1774	0.1745	0.1852	0.1790	2.86×	71.6%
N = 8	0.1310	0.1133	0.1195	0.1213	4.23×	52.8%
N = 16	0.1059	0.1183	0.1541	0.1261	4.06×	25.4%
N = 32	0.1003	0.1031	0.1024	0.1019	5.03×	15.7%

Q2.1: Scaling Saturation & Hardware Contention Analysis

Speedup scales reasonably well up to $N = 8$, which exactly matches the M2's 8 physical cores (4 Performance + 4 Efficiency). Efficiency drops steadily even before saturation, from 100% at $N=1$ to 52.8% at $N=8$, because the M2's cores are not uniform: the OS scheduler places some threads on fast P-cores and others on slower E-cores, so the average per-thread contribution decreases even while total cores are still available.

At $N = 16$, speedup actually drops slightly versus $N = 8$ (4.06x vs 4.23x), even though efficiency (25.4%) still appears to scale. This is the inflection point: past 8 threads, the 8 physical cores become oversubscribed, forcing the OS to time-slice multiple threads per core. The added context-switching overhead outweighs any additional parallel gain, a textbook oversubscription (contention) wall.

The slight uptick at $N = 32$ (5.03x) is likely workload-imbalance noise rather than genuine scaling: trial-division cost is not uniform across the number range, so slicing the range into more, smaller chunks changes how that imbalance is distributed, not a sign of real additional parallel capacity, since only 8 physical cores exist.

TASK 3: The Unsynchronized Shared Counter & Race Condition Trap (25 Points)

Run	Measured Output	Error (10^7 - Act)	Run	Measured Output / Error
#1	2,845,602	7,154,398	#6	1,828,811 / 8,171,189
#2	3,265,015	6,734,985	#7	3,182,890 / 6,817,110
#3	2,856,151	7,143,849	#8	2,873,634 / 7,126,366
#4	2,616,159	7,383,841	#9	3,013,947 / 6,986,053
#5	3,372,642	6,627,358	#10	3,254,597 / 6,745,403

Average measured across 10 runs: approximately 2,910,945 (roughly 71% of increments lost to the race).

Q3.1: Read-Modify-Write (RMW) Window

`shared_counter++` is not a single CPU instruction; it decomposes into three separate machine steps: Load the current value from memory into a register, Add 1 to that register value, and Store the updated value back to memory. When multiple threads run across the M2's 8 cores, two or more cores can load the same value of `shared_counter` before either has stored its incremented result back. For example, if the counter is at 500, Core A loads 500, and before it stores 501, Core B also loads 500; both cores compute 501 and store it, but two increments happened while the counter only advanced by one. This lost-update pattern happens on the vast majority of the 10 million increment attempts, which is why the measured total (~2.9M) is far below the expected 10M.

Q3.2: The Synchronization Penalty

Unlocked ≈ 17.3 ms (average of 10 runs) vs. Locked = 353.1 ms (~20x slower)

The synchronized version is dramatically slower because `pthread_mutex_lock/unlock` forces serialization: only one thread can execute the increment at any moment, turning a parallel workload back into an effectively sequential one. Each lock/unlock call also involves a synchronization primitive that is far more expensive than a plain memory read/write, and with 10 million lock acquisitions total, that per-operation overhead compounds massively.

TASK 4: Empirical Amdahl Verification vs. Gustafson Scaling Horizon (15 Points)

1. Parallel Fraction Derivation (p)

$$T_1 = 0.5125 \text{ s} \quad | \quad T_2 = 0.3147 \text{ s}$$

$$p = 2 \times (T_1 - T_2) / T_1 = 2 \times (0.5125 - 0.3147) / 0.5125 = 0.3956 / 0.5125 \approx 0.772$$

2. Amdahl Theoretical Speedup Ceiling

$$\text{Sequential fraction } (1 - p) = 0.228$$

$$S_{\max} = 1 / (1 - p) = 1 / 0.228 \approx 4.38\times$$

3. Amdahl 64-Core Projection

$$S_{64} = 1 / [(1 - p) + (p / 64)] = 1 / (0.228 + 0.01206) = 1 / 0.24006 \approx 4.17\times$$

4. Gustafson Scaled Speedup (Weak Scaling)

$$SGustafson = (1 - p) + (p \times 64) = 0.228 + 49.408 \approx 49.64\times$$

Q4.1: Strong vs. Weak Scaling Analysis

Amdahl's Law assumes a fixed problem size (strong scaling): as more cores are added, the same amount of work is divided into smaller pieces, but the 22.8% sequential portion does not shrink. It stays a fixed, unparallelizable chunk of time regardless of core count, which is why S64 (4.17x) barely improves over the theoretical maximum (4.38x) - once the parallel portion is split thin enough among many cores, the fixed sequential fraction dominates the runtime, creating a hard ceiling.

Gustafson's Law assumes the problem size grows with the number of cores (weak scaling): instead of splitting the same task into smaller pieces, 64x more total work is done in roughly the same wall-clock time, since each core is given its own full-sized chunk of new work rather than a sliver of the original chunk. Because the sequential portion does not need to scale up with the problem, the parallel portion completely dominates as cores increase, producing the near-linear 49.64x speedup.

In short: Amdahl answers how much faster the exact same task can finish (bounded by the fixed sequential part), while Gustafson answers how much more work can be completed in the same time (bounded only by available cores). Same algorithm, same p, but the two laws describe opposite goals, which is why they produce such different projections at 64 cores.